Instrukcja uruchomienia i testowania

Instrukcja krok po kroku: jak wdrożyć system na lokalnym klastrze Minikube, jak go przetestować, jak uruchomić wariant GitOps (ArgoCD) oraz profil Prometheus Operator. Wszystkie polecenia wykonuje się z katalogu głównego repozytorium.

> **Status weryfikacji:** wszystkie kroki tej instrukcji zostały wykonane od zera na czystym klastrze Minikube (osobny profil) w dniu 2026-11-01; wynik: wdrożenie, dostęp, audyt, test (dwa kolejne przebiegi), ArgoCD z self-heal, profil operatora i sprzątanie działają zgodnie z opisem.

1. Wymagania

Narzędzie	Do czego	Sprawdzenie
Docker	sterownik Minikube i budowa obrazu	`docker version`
Minikube	lokalny klaster Kubernetes	`minikube version`
kubectl (v1.28+)	obsługa klastra, `kubectl kustomize`	`kubectl version --client`
Helm (tylko profil operatora)	instalacja `kube-prometheus-stack`	`helm version`
Python 3 + `httpx` (tylko test lokalny poza klastrem)	skrypt testowy	`python3 -c "import httpx"`

Zalecane zasoby Minikube: 4 CPU i 6 GB RAM dla samego stosu; 6 CPU i 12 GB RAM, jeśli dodatkowo działają ArgoCD i profil operatora.

```
minikube start --driver=docker --cpus=6 --memory=12288
```

2. Wdrożenie (jedno polecenie)

```
./scripts/deploy_minikube.sh cpu-demo        # profil demonstracyjny (mock Triton, bez GPU)
# ./scripts/deploy_minikube_gpu.sh             # prawdziwy Triton + vLLM na GPU (rozdz. 2a)
```

Skrypt: uruchamia Minikube (jeśli nie działa), włącza dodatki `ingress` i `metrics-server`, buduje obraz `mcp-agent:local`, tworzy sekrety (`scripts/bootstrap_secrets.sh`), stosuje overlay `k8s/overlays/local-cpu` i czeka na gotowość podów.

> **Uwaga:** zaraz po włączeniu dodatku `ingress` kontroler potrzebuje chwili. Jeśli `kubectl apply` zgłosi błąd webhooka `validate.nginx.ingress.kubernetes.io ... connection refused`, odczekaj i powtórz: `kubectl -n ingress-nginx rollout status deploy/ingress-nginx-controller && kubectl apply -k k8s/overlays/local-cpu`

Sprawdzenie:

```
kubectl -n inference-monitoring get pods,pvc,svc,ingress
```

Oczekiwane: 5 podów `Running` (`prometheus-0`, `grafana`, `alertmanager`, `mcp-agent`, `triton-server`), dwa PVC `Bound` (`prometheus-storage-prometheus-0` 2Gi, `grafana-storage` 1Gi).

2a. Wariant GPU (prawdziwy Triton + vLLM)

Wymagania hosta: sterownik NVIDIA, `nvidia-container-toolkit` ze skonfigurowanym runtime w Dockerze oraz pliki modelu w `models/finance-qwen/1/` (`model.safetensors`, `config.json`, pliki tokenizera; nie są w repozytorium).

```
sudo nvidia-ctk runtime configure --runtime=docker --set-as-default && sudo systemctl restart
docker   # jednorazowo
docker run --rm --gpus all ubuntu nvidia-smi -L
# sprawdzenie
./scripts/deploy_minikube_gpu.sh
```

Skrypt tworzy osobny profil Minikube `gpu` (GPU przekazane do węzła, wtyczka urządzeń NVIDIA), kopiuje model na węzeł, pobiera obraz Tritona z vLLM (kilkanaście GB przy pierwszym uruchomieniu), buduje obraz agenta i stosuje overlay `local-gpu`. Konfiguracja silnika (`config.pbtxt`, `model.json`) jest zasiewana z ConfigMap na wspólny wolumen modelu, który współdzielą Triton i agent (agent edytuje `model.json`, a Triton przeładowuje silnik).

Weryfikacja i test na GPU:

```
kubectl describe node | grep nvidia.com/gpu
kubectl -n inference-monitoring exec deploy/triton-server -c triton-server -- nvidia-smi
TEST_PROFILE=gpu LOADGEN_IMAGE=mcp-agent:local ./scripts/run_in_cluster_test.sh
```

Profil `TEST_PROFILE=gpu` wysyła żądania na endpoint `/generate`, startuje od `max_num_seqs = 4` i porównuje opóźnienie end-to-end mierzone w klastrze (Triton z vLLM jest modelem „decoupled”, więc jego liczniki nie mierzą czasu generowania). Wdrożenie GPU przez GitOps: `kubectl apply -f k8s/gitops/argocd-application-gpu.yaml` (zamiast `argocd-application.yaml`).

3. Dostęp do usług

Ingress obsługuje host `inference.local` z TLS (certyfikat self-signed, przeglądarka pokaże ostrzeżenie).

```
echo "$(minikube ip) inference.local" | sudo tee -a /etc/hosts
# Minikube (sterownik docker na Linuksie): kontroler Ingress to usługa NodePort dostępna pod
adresem z polecenia: minikube ip
# Gdy adres Minikube jest nieosiągalny z hosta (macOS/Windows), użyj w osobnym terminalu
`minikube tunnel`
# i wpisu 127.0.0.1 w /etc/hosts.
```

Bez uprawnień `sudo` (lub bez edycji `/etc/hosts`) wystarczy `curl --resolve`, np.:

```
curl -sk --resolve inference.local:443:$(minikube ip)
https://inference.local/grafana/api/health
```

Usługa	Adres	Uwagi
Grafana	`https://inference.local/grafana/`	login `admin`, hasło poniżej
Triton	`https://inference.local/v2/health/ready`	mock w profilu `cpu-demo`, vLLM w profilu GPU
MCP Agent	`https://inference.local/mcp/health`	`/mcp/...` jest przepisywane na `/...`
Prometheus	`kubectl -n inference-monitoring port-forward svc/prometheus-service 9090:9090`	bez uwierzytelniania, dlatego bez Ingress
Alertmanager	`kubectl -n inference-monitoring port-forward svc/alertmanager-service 9093:9093`	j.w.

Hasło Grafany jest losowe i nie leży w repozytorium:

```
kubectl -n inference-monitoring get secret grafana-secret -o jsonpath='{.data.admin-password}'
| base64 -d; echo
```

Skrót do wszystkich port-forwardów: `./scripts/start_port_forward.sh &`.

4. Testy

4.1. Audyt bezpieczeństwa (RBAC, non-root, NetworkPolicy, quota)

```
./scripts/verify_security.sh
```

Oczekiwane: `prometheus-sa` może czytać pody (`yes`), nie może ich usuwać (`no`) ani czytać Secrets (`no`); wszystkie pody mają `NonRoot: true`; wylistowane są NetworkPolicy, ResourceQuota i LimitRange; w manifestach nie ma jawnego hasła.

4.2. Test obciążeniowy, alertu i hot-reload (w klastrze)

```
LOADGEN_IMAGE=mcp-agent:local ./scripts/run_in_cluster_test.sh
```

Skrypt tworzy Job `loadgen` (z `NetworkPolicy` tylko do Tritona, agenta i Prometheusa), czeka na zakończenie, wypisuje log i zapisuje `test-output.log` oraz `test-report.json` (zmienna `OUT_DIR` zmienia katalog). Kod wyjścia jest różny od 0, jeśli którekolwiek kryterium nie przejdzie.

Fazy: **CALM** (20 żądań/s), **STORM** (seria 400 równoczesnych żądań na sekundę, zmienna `STORM_RPS`), rekonfiguracja `/reconfigure` (`max_num_seqs` 32 → 512, hot-reload), **RESOLUTION** (ten sam ruch). Kryteria PASS i interpretacja są w `docs/ZADANIE_3_IMPLEMENTACJA_I_SPRAWOZDANIE.md`, rozdz. 5.

Po teście możesz obejrzeć wykresy w Grafanie (dashboard *Autonomous Inference Cluster*) i aktywne alerty w Prometheusie (`/alerts`).

Test można uruchamiać wielokrotnie: na początku sam przywraca stan bazowy (mock: endpoint `/_mock/reset`, `max_num_seqs` = 32; prawdziwy silnik: ustawienie `max_num_seqs` przez agenta i poczekanie na przeładowanie modelu).

4.3. Walidacja manifestów lokalnie (bez klastra)

```
for d in k8s/base k8s/overlays/cpu-demo k8s/overlays/gpu k8s/overlays/local-cpu
k8s/overlays/local-gpu k8s/addons/operator-stack; do
  kubectl kustomize "$d" > /dev/null && echo "$d OK"
done
```

To samo, plus `kubeconform`, `yamllint`, `shellcheck` i skan Trivy, wykonuje potok CI (rozdz. 7).

5. GitOps z ArgoCD (zadanie 4)

```
kubectl create namespace argocd
kubectl apply -n argocd --server-side -f https://raw.githubusercontent.com/argoproj/argo-
cd/stable/manifests/install.yaml
kubectl -n argocd rollout status deploy/argocd-server

./scripts/bootstrap_secrets.sh                        # sekrety muszą istnieć przed pierwszą
synchronizacją
kubectl apply -f k8s/gitops/argocd-application.yaml
kubectl -n argocd get app inference-monitoring-app    # oczekiwane: Synced / Healthy
```

ArgoCD pobiera overlay `k8s/overlays/cpu-demo` z gałęzi `main` i obraz `ghcr.io/shengan01/mcp-agent` (pakiet jest publiczny). Jeśli stos był wcześniej wdrożony skryptem `deploy_minikube.sh`, ArgoCD przejmuje istniejące zasoby i podmienia obraz agenta z lokalnego na rejestrowy. Tuż po utworzeniu aplikacji stan `Progressing` trwa zwykle kilkadziesiąt sekund. Uwaga: ArgoCD ma pierwszeństwo przed lokalnymi zmianami (self-heal) – zmiany manifestów trzeba wypchnąć do Git.

Test self-heal:

```
kubectl -n inference-monitoring delete svc alertmanager-service
kubectl -n inference-monitoring scale deploy/grafana --replicas=0
kubectl -n inference-monitoring get svc alertmanager-service deploy/grafana    # po
kilkudziesięciu sekundach oba wracają
```

Pełny przepływ: `git push` → GitHub Actions (testy, obraz, skan) → job `gitops-bump` zapisuje nowy tag obrazu w overlayach → ArgoCD wykrywa zmianę w Git i wdraża.

6. Profil Prometheus Operator (Helm, opcjonalny)

```
OPERATOR_PROFILE=true ./scripts/bootstrap_secrets.sh      # sekret grafana-admin w namespace
monitoring
kubectl apply -f k8s/gitops/argocd-application-operator.yaml
kubectl -n argocd get app kube-prometheus-stack operator-stack-resources
```

Weryfikacja:

```
kubectl -n monitoring port-forward svc/kps-prometheus 19090:9090 &
curl -s localhost:19090/api/v1/targets | grep -o '"scrapePool":"serviceMonitor/inference-
monitoring[^"]*"' | sort -u   # .../mcp-agent-config/0 i .../triton/0
curl -s localhost:19090/api/v1/rules | grep -o 'inference_alerts'
```

Bez ArgoCD, ręcznie przez Helm:

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm upgrade --install kps prometheus-community/kube-prometheus-stack -n monitoring --create-
namespace \
  --version 58.2.2 -f k8s/addons/operator-stack/helm-values.yaml
kubectl apply -k k8s/addons/operator-stack
```

7. Potok CI/CD (GitHub Actions)

Plik `.github/workflows/ci-cd.yaml`, uruchamiany po każdym pushu i pull requeście na `main`:

Job	Co robi
`lint-and-validate`	`yamllint`, `shellcheck`, kompilacja skryptów Pythona, `kubectl kustomize` każdego overlaya + `kubeconform`
`security-audit`	Trivy na manifestach (CRITICAL blokuje, HIGH idzie do SARIF)
`integration-test`	klaster `kind`, wdrożenie `local-cpu`, `verify_security.sh`, test w klastrze; artefakt `test-report`
`build-scan-push` (tylko `main`)	budowa obrazu, skan Trivy (naprawialne HIGH/CRITICAL blokują), push do `ghcr.io`
`gitops-bump` (tylko `main`)	zapis nowego tagu obrazu w Git (commit `[skip ci]`)

Wynik: zakładka *Actions* w repozytorium. Raport z testu pobierzesz z artefaktu `test-report` danego przebiegu.

8. Sprzątanie

```
kubectl delete -f k8s/gitops/argocd-application-operator.yaml        # jeśli używano profilu
operatora
kubectl delete -f k8s/gitops/argocd-application.yaml                 # jeśli używano ArgoCD:
usuwa zasoby aplikacji razem z namespace
kubectl delete -k k8s/overlays/local-cpu                             # wdrożenie bez ArgoCD
kubectl delete namespace monitoring argocd                           # profil operatora i
ArgoCD
minikube stop                                                        # zatrzymanie klastra,
dane zostają
```

9. Najczęstsze problemy

Objaw	Przyczyna / rozwiązanie
Błąd webhooka `validate.nginx.ingress.kubernetes.io` przy `kubectl apply`	kontroler Ingress jeszcze nie wstał; poczekaj na `rollout status` i powtórz
`Grafana` zwraca 404 pod `/`	Grafana jest serwowana pod `/grafana/`
`curl: (6) Could not resolve host: inference.local`	brak wpisu w `/etc/hosts` (rozdz. 3)

Objaw	Przyczyna / rozwiązanie
Pod `mcp-agent` w `ImagePullBackOff` w profilu `cpu-demo`	pakiet `ghcr.io/shengan01/mcp-agent` musi być publiczny; lokalnie użyj overlaya `local-cpu`
Pody `Pending` z powodu quota	`kubectl -n inference-monitoring describe resourcequota`
Test STORM nie pokazuje degradacji po zmianie manifestów mocka	ConfigMap `mock-triton-script` nie został zastosowany; wykonaj `kubectl apply -k k8s/overlays/local-cpu` i `kubectl -n inference-monitoring rollout restart deploy/triton-server`
Grafana z chartu nie startuje	brak sekretu `grafana-admin`: `OPERATOR_PROFILE=true ./scripts/bootstrap_secrets.sh`